claude-windows / 56b7f849-5d56-4770-93e0-b3b4da943700

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\56b7f849-5d56-4770-93e0-b3b4da943700\subagents\workflows\wf_49729ae7-962\agent-a776c8e8686670652.jsonl`
- SHA-256: `b5f4b7e7d2b1e0fc1e031b5c447866203138a0e64888f9204a842ce5d699c5aa`
- Source modified: `2026-06-07T08:14:07+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_49729ae7-962`
- session_id: `56b7f849-5d56-4770-93e0-b3b4da943700`

Transcript

1. user (2026-06-07T08:13:58.711Z)

You are a USER of a sports-video tool who has read ONLY its README/FAQ ? you have NEVER seen the source code. A processing run looks wrong and the tool wrote a per-video report. Read EXACTLY these two files and NOTHING else (do not open source code or any other file):
1. The README/FAQ: C:\Users\avidu\Projects\sports-data-collector\docs\INGESTION_PIPELINE.md
2. The tool's technical report for the suspicious run:
C:\Users\avidu\AppData\Local\Temp\obseval\reports\collector_overlap.report.md

Using ONLY those two, tell me: what went wrong, exactly what you'd do to fix it, and which FAQ section (by heading text or Q-number) you used. Be honest about whether you could truly self-debug this from the report alone, and list anything in the report that was confusing, ambiguous, or under-explained.

2. assistant (2026-06-07T08:14:00.432Z)

I'll read both files to understand the issue and provide the analysis you requested.

3. user (2026-06-07T08:14:00.481Z)

~~~~
# Rally Ingestion Pipeline ? End-to-End FAQ

How a vendor's rally annotations become training/eval ground truth, and how we
validate each delivery against our conventions. This is the operational companion
to the annotation spec (`docs/annotation/ROORA_BRIEF.md` + `ANNOTATION_GUIDE.md`).

```
  vendor delivery                  OUR side (collector)                 modeling (indexer)
 ??????????????????   convert    ?????????????????  validate  ????????????????
 ? CVAT export    ? ???????????? ? canonical CSV ? ?????????? ? report .md   ?
 ? (boxes + attrs)?              ? (per rally,   ?            ? report .json ?
 ?   OR           ?              ?  decimal secs)?            ? review .csv  ???? human verifies
 ? canonical CSV  ?              ?????????????????            ????????????????  flagged rallies
 ??????????????????                    ?                             ?
                                        ?  import-local (validated)  ?
                                        ?                     golden labels
                                       ?????????????????  export   ????????????????
                                       ? SQLite (rally ? ????????? ? <id>.csv     ?  start,end (indexer GT)
                                       ?  tables)      ?           ? manifest.json?
                                       ?????????????????           ????????????????
                                                                         ?
                                                                 slice clips / calibrate /
~~~~

train
```

The four commands (in order)

```bash
# 1. Normalise a CVAT export ? our canonical per-rally CSV (skip if the vendor sent a CSV)
python -m src.cli.curate convert-cvat --input job.xml \
    --video-path match.mp4 --out canonical.csv --issues issues.txt

# 2. Validate the delivery ? report + annotatable review sheet (the QA gate)
python -m src.cli.curate validate-delivery --input job.xml \
    --video-path match.mp4 --video-id pilot_badminton_01 --out-dir data/validation
#   ...or validate a CSV directly:
python -m src.cli.curate validate-delivery --csv canonical.csv --video-id pilot_badminton_01

# 3. After human review, import the (corrected) CSV into the DB
python -m src.cli.curate import-local --sport badminton \
    --video-path match.mp4 --annotations-path canonical.csv --fps 59.94

# 4. Export indexer-ready ground truth (start,end CSVs + manifest)
python -m src.cli.curate export --out-dir data/eval_export
```

FAQ

What delivery formats do we accept?
Both. A **CVAT export** (image- or track-mode XML ? rally fields carried as box attributes) is normalised by `convert-cvat`; an **already-canonical CSV** is taken as-is. Vendors may keep working in CVAT ? we do **not** require them to hand-author CSVs.

What is the "canonical" CSV?
One row per rally:
`rally_number, start_mmss, end_mmss, start_time, end_time, shots_count, ending_reason, last_shot_outcome, score_before, score_after, notes`.
`start_time`/`end_time` are **decimal seconds**; `import-local` reads those.

How are timestamps handled? (the fps thing)
We recompute each rally's interval from its **frame range × the true fps** (`seconds = frame / fps`). A vendor's CVAT time attributes are **ignored** ? a step-sampled job that assumes 30 fps writes systematically wrong times. fps comes from `--fps` or ffprobe of `--video-path`. `mm:ss` is also accepted everywhere.

What does validation check?
code	severity	meaning
`overlap`	**blocker**	two rallies' intervals overlap (`end[N] > start[N+1]`)
`zero_duration`	**blocker**	`end <= start` (single-sample / stray box)
`frame_out_of_range`	high	a box frame is past the video's last frame
`shot_density`	medium	long rally with implausibly few shots ? likely an **inflated span** (often hides a merged rally)
`possible_missed_rally`	medium	a long unlabeled gap between two rallies ? check for an **omitted rally**
`ending_reason_vocab`	medium	`ending_reason` outside the agreed set
`outcome_vocab`	low	`last_shot_outcome` outside `{in,net,out,n_a}`

Blocker vs. review ? what's the gate?
`validate-delivery` exits **non-zero if any blocker exists** (so it can gate a bulk loop / CI). Non-blocker issues don't stop the gate but populate the **review sheet** for the human pass. `import-local` independently rejects overlaps/zero-duration, so
```

bad ground truth can't slip into the DB.

### How are *missed rallies* caught? (the worst error)
Two complementary checks, because a miss shows up two ways:
1. **Clean omission** (a real hole in the timeline) ? `possible_missed_rally` (gap heuristic).
2. **Merged/inflated** (the missed rally got swallowed into a neighbour's box, leaving no gap) ? `shot_density` flags the over-long, under-shot rally, and the human finds the second rally inside it.

Neither is a guarantee ? they **surface regions to review**, they don't certify completeness. (A future enhancement: a motion second-opinion over the raw video.)

### What's the review sheet?
`<id>_review.csv` ? one row per rally with `auto_flags` (which checks fired) plus blank `VERIFIED / true_start_mmss / true_end_mmss / true_ending_reason / true_last_shot_outcome / reviewer_comment` columns. Open it next to the video (timestamp overlay on) and fill the flagged rows. This is the human gate that turns a delivery into trusted golden labels.

### Why two ending fields (`ending_reason` + `last_shot_outcome`)?
They're orthogonal: `ending_reason` is *why* the point ended (winner / forced_error / unforced_error / service_fault / let / other); `last_shot_outcome` is *where* the last shot went (in / net / out / n_a). A shuttle that flew long is `forced_error/unforced_error` + `out` ? both kept, no information lost.

### How do I onboard a brand-new video?
1. Get fps into the intake manifest (we provide it to the vendor).
2. `convert-cvat` (if CVAT) ? `validate-delivery` ? read the report, fill the review sheet for flagged rallies ? correct the CSV ? `import-local` ? `export`.
3. The same suite runs for every video, so quality stays consistent at scale.

### How do I pull a single rally out to eyeball it?
The slicer (indexer):
```bash
python -m backend.tools.rally_slicer --video match.mp4 \
    --labels golden_labels.csv --rallies 7,16 --out-dir output/rally_clips
```
Clips are padded to match the highlight compiler (0.5s lead / 1.0s tail).

Does coarse frame sampling hurt boundaries?
Not in practice: ~0.5s sampling slack is absorbed by the highlight compiler's clip padding, so we don't flag sampling density.

Will a non-commercial license block export?
The video is **kept** in the DB regardless of license, but the default `export` is **commercial-only** ? a non-commercial source (e.g. `CC-BY-NC-SA`) is *excluded* from the eval/training set unless you pass `--include-noncommercial`. So if a video "goes missing" downstream, check its `is_commercial` flag first. (The observability report raises a `license_noncommercial` warning for exactly this case.)

What is the observability report? (`*.report.json` / `*.report.md` / `*.summary.md`)
Every `validate-delivery`, `convert-cvat`, and `import-local` run now also writes a per-video **observability report** (next to the video for `import-local`; into the `--out-dir` / next to the canonical CSV otherwise):
- **`<id>.report.json`** ? machine-readable: every stage, metric, and finding. Diff two runs to see what changed between deliveries.
- **`<id>.report.md`** ? the **technical** report: parse / validation / registration stages plus **findings** (one per delivery issue, each citing this FAQ). Read this to debug a delivery.
- **`<id>.summary.md`** ? a **delivery summary** for the vendor / data team: the rally count and coverage plus the plain-language list of rallies to review ? no internal codes or severities.

This is in addition to the existing `<id>_report.md` / `_report.json` / `_review.csv` (the human review sheet), which are unchanged.

4. user (2026-06-07T08:14:00.577Z)

```
# Sports Data Collector ? Delivery / Ingestion Report

## Run

| Field | Value |
| --- | --- |
| Tool | sports-data-collector |
| Video id | pilot_overlap (human_id) |
| Generated | 2026-06-07T08:10:51Z |
| Schema | 1.0 |

## Video

| Field | Value |
| --- | --- |
| Duration | 10m00s |
| FPS | 59.94 fps (declared) |

## Processing stages

**parse** ? `ok` (0.0s)
  - rallies_parsed: 2
  - _source_format_: cvat
  - _fps_: 59.94
  - _sampling_step_: None

**validation** ? `error` (0.0s)
  - issues_blocker: 1
  - issues_high: 0
  - issues_medium: 0
  - issues_low: 0
  - n_blockers: 1
  - _gate_: BLOCKED

## Findings & issues

- [ERROR] **overlap** ? Rally 1 (ends 7.00s) overlaps Rally 2 (starts 6.00s) by 1.00s ? rallies must be disjoint.
  ? see **docs/INGESTION_PIPELINE.md#what-does-validation-check**
  ? evidence: rally_number=2

## Verdict

**FAIL**

```

5. assistant (2026-06-07T08:14:01.969Z)

Now I have both files. Let me analyze what went wrong.

6. user (2026-06-07T08:14:07.879Z)

Structured output provided successfully